

Week-1



Basics of DSA & Foundations

- Introduction to DSA
- Problem Solving Approach
- Time and Space Complexity
- Notations (Big-O, Ω , Θ)
- Aspects of Algorithms
- Extra: Introduction to Pseudocode & Flowcharts

Data Structures & Algorithms (DSA) – Basics & Foundations

1. Introduction to DSA

What is DSA?

- **Data Structure (DS):**

A way to **store, organize, and manage data** efficiently.

Examples: Array, Stack, Queue, Linked List, Tree, Graph, Hash Table.

- **Algorithm (A):**

A step-by-step **procedure to solve a problem** or perform a task.

Example: Sorting numbers, searching an element.

👉 Together, **DSA provides optimized ways of storing data and solving problems.**

Why study DSA?

- Helps write **efficient** programs.
 - Backbone for **interviews and placements**.
 - Essential in **real-world applications**:
 - Social Media → Graphs (friends, followers)
 - Google Maps → Graph Algorithms (shortest path)
 - E-commerce → Searching & Sorting
-

2. Problem Solving Approach

Steps in Problem Solving:

1. Understand the Problem

- Input/Output requirements
- Constraints

2. Plan the Solution

- Choose the right **algorithm**
- Select the best **data structure**

3. Write Pseudocode / Flowchart

- Logical representation of the solution

4. Implement Code

- Translate pseudocode into programming language

5. Analyze the Solution

- Time Complexity & Space Complexity

6. Test with Edge Cases

- Large inputs, zero values, negative values

◆ Example:

Problem – Find the largest number in an array.

- Input: [10, 25, 3, 89, 56]
- Output: 89

Pseudocode

```
max = arr[0]
for i from 1 to n-1
    if arr[i] > max
        max = arr[i]
return max
```

3. Time and Space Complexity

Time Complexity

- Measures **how fast an algorithm runs** (time grows with input size).
- Expressed as a function of n (input size).

Example:

Linear Search in array of n elements $\rightarrow O(n)$

Space Complexity

- Measures **how much memory an algorithm uses**.
- Includes:
 - Input space
 - Temporary variables
 - Recursion stack

Example:

Binary Search uses $O(1)$ extra space.

Recursive Binary Search uses $O(\log n)$ stack space.

4. Notations (Big-O, Ω , Θ)

(a) Big-O Notation – Upper Bound

- Represents **worst-case** performance.
- Example: Linear Search → $O(n)$

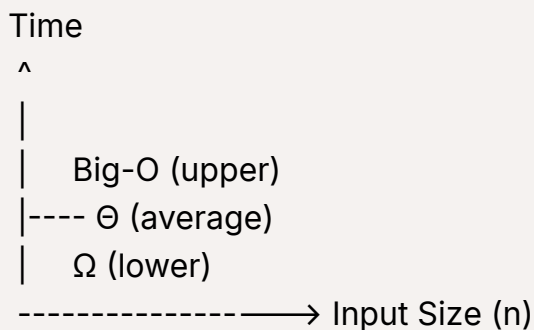
(b) Omega (Ω) – Lower Bound

- Represents **best-case** performance.
- Example: Linear Search → $\Omega(1)$ (if element is first)

(c) Theta (Θ) – Tight Bound

- Represents **average-case** (or exact bound).
- Example: Linear Search → $\Theta(n)$

Graphical Representation



5. Aspects of Algorithms

When analyzing algorithms, we look at:

1. **Correctness** – Does it always give the right answer?
2. **Finiteness** – Algorithm must end after finite steps.
3. **Determinism** – Each step must be clear and unambiguous.
4. **Efficiency** – Good time & space complexity.
5. **Generality** – Works for a wide range of inputs.

6. Introduction to Pseudocode & Flowcharts

Pseudocode

- A **high-level, language-independent** way to write an algorithm.
- Uses simple English + programming constructs.

Example: Factorial of n

```
function factorial(n)
  if n == 0
    return 1
  else
    return n * factorial(n-1)
```

Flowcharts

- A **visual diagram** to represent the flow of an algorithm.
- Uses **shapes**:
 - **Oval** → Start/End
 - **Parallelogram** → Input/Output
 - **Rectangle** → Process/Calculation
 - **Diamond** → Decision

Example: Factorial Flowchart

```
[Start]
|
Input n
|
fact = 1, i=1
|
[ i <= n ? ]
/      \
Yes      No
```

```
|      |  
fact=fact*i   Print fact  
i=i+1      |  
|      |  
-----  
|  
[End]
```